

AIFlow Architecture: Layer-by-Layer Breakdown

The AIFlow architecture follows a decoupled hexagonal/layered pattern, moving top-to-bottom from user interaction to low-level system operations.

1. Interface Layer (User-Facing)

This is the entry point of the application where input is captured and results are presented.

- **CLI Controller (clap v4)**: Parses user arguments, flags, and commands typed into the terminal.
- **Terminal Formatter (comfy-table / owo-colors)**: Formats the application's responses with colors, tables, and styled text for clear visual presentation in the console.

2. Application & Service Layer (Orchestration)

Acts as the intermediate layer that routes CLI requests to specific business logic handlers.

- **Status Service**: Handles system status checks and formats the current project health.
- **Project Discovery Service**: Identifies and initializes active projects and directory structures.
- **Next Action Engine**: Determines what step or command the user should take next.
- **Doctor & Health Linter**: Diagnoses potential issues, misconfigurations, or broken state files.
- **Prompt Scaffold Service**: Generates structured prompts for downstream usage.

3. Domain & State Engine (Core Business Logic)

Houses the core domain knowledge and manages state transformations without depending on lower-level infrastructure details.

- **Workflow State Machine (FSM)**: Manages phase transitions, step progression, and operational constraints.
- **State & Evidence Aggregator**: Gathers signal inputs across the system to form a single source of truth for current state.
- **Domain Models**: Core data types representing primary entities—**Project**, **Phase**, **Task**, and **Role**.

4. Heuristic & Detection Engine (Signal Analyzers)

Analyzes workspace signals and environment artifacts to infer status automatically.

- **Git Signal Detector**: Evaluates repository context (branches, uncommitted changes, history).
- **Test Runner Detector & Parser**: Identifies test suites, runs test routines, and parses test outputs.

- **Documentation Staleness Analyzer:** Detects outdated docs by cross-referencing file changes and timestamps.
- **Artifact Spec Inspector:** Validates and inspects project metadata files and artifacts.

5. Infrastructure Adapters (Low-Level System I/O)

Converts domain requests into hardware, storage, and external process operations.

- **Git Adapter (`git2` / CLI fallback):** Interacts directly with Git via `libgit2` bindings or shell invocation.
- **Filesystem Adapter (`walkdir` / `ignore`):** Traverses workspace directories while respecting `.gitignore` rules.
- **Artifact Storage Manager (`serde_yaml` / `pulldown_cmark`):** Reads and parses structured YAML data and Markdown artifacts.
- **Role & Provider Registry:** Manages AI role definitions and provider configuration mappings.

Data Flow Summary

1. **Request Intake:** A user runs a command handled by the **CLI Controller**.
2. **Service Dispatch:** The CLI forwards the command to the appropriate **Service** (e.g., Status, Doctor, Next Action).
3. **State & Signal Aggregation:** The service queries the **State & Evidence Aggregator**, which triggers **Heuristic Detectors** to analyze workspace state.
4. **Adapter I/O Execution:** Detectors invoke low-level **Adapters** to read files, query Git, or inspect specs.
5. **Domain Processing:** Raw evidence flows into the **Workflow State Machine (FSM)** to evaluate domain logic and rules.
6. **Response Presentation:** The result is passed back to the **Terminal Formatter** to display clean output to the user.